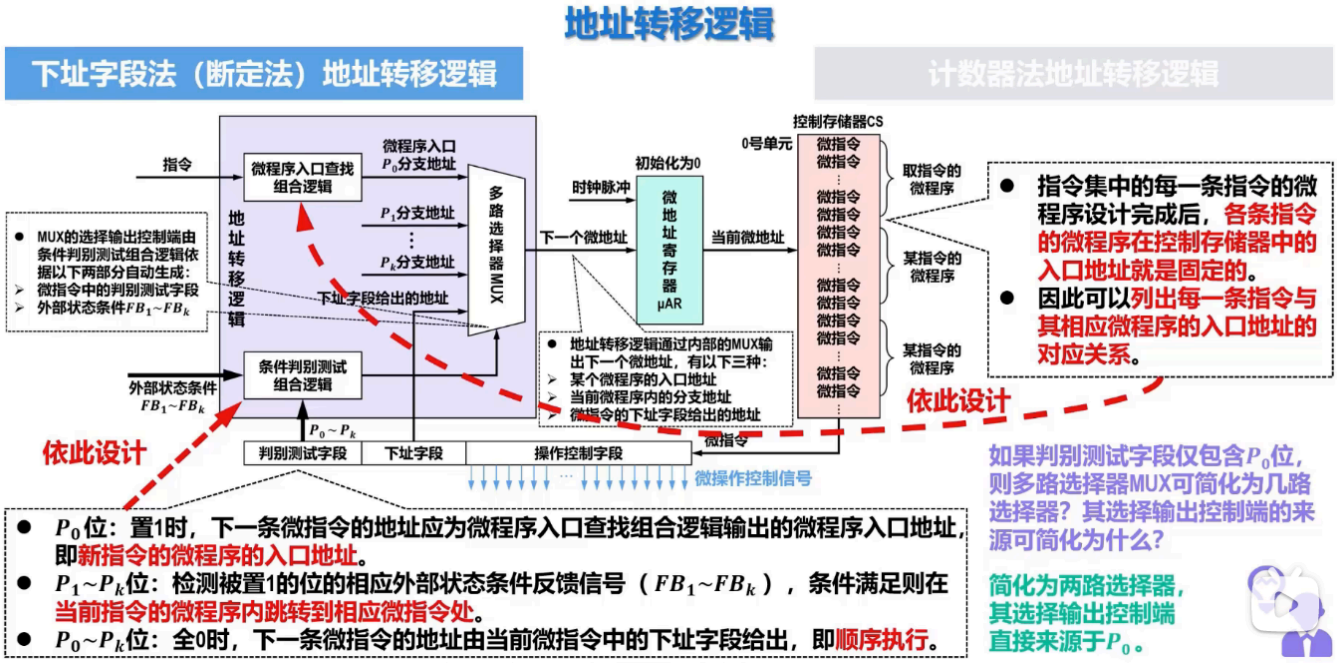
一、下地址字段法



先定位一下：紫色方框——**地址转移逻辑**，这张图就是把它“拆开给你看内部构造”。拆开后发现，里面其实只有三样东西：

> **两个组合逻辑电路 + 一个多路选择器（MUX）**
> 它全部的任务只回答一个问题：**“下一个微地址是多少？”**

1.1 核心思想：下一微地址只有 3 种可能的来源

不管微程序多复杂，某条微指令执行完，“下一站”无非这三种：

来源	白话解释	由谁触发
① 某个**微程序的入口地址**	“换节目”——刚取完一条新机器指令，跳到它的微程序开头	$P_0 = 1$
② 当前微程序内的**分支地址**	“if 跳转”——本微程序内部按条件跳到某一步	$P_i = 1$ 且条件 FB_i 成立
③ **下址字段**给出的地址	“下一步”——老老实实顺序执行	P_0~P_k 全 0
既然候选地址有限、且事先全部已知，那用一个“多路选择器 MUX”来挑选再合适不过——就像电视机的信号源切换：HDMI1、HDMI2、TV 三路画面都接好了，遥控器按哪个键就显示哪一路。图中的 MUX 就是这个“切换器”，把几路候选地址择一输出，作为“下一个微地址”送进 μAR		

1.2 两个组合逻辑电路的作用

1️⃣ 微程序入口查找组合逻辑 —— “菜谱目录页”

看右侧灰色框里的两行字，这是它能存在的前提：

> 微程序设计完成后，**各条指令的微程序在 CS 中的入口地址就固定了** → 因此可以列出“指令 ↔ 入口地址”的对应关系表
> 既然是一张**固定不变的表**，那就能用纯硬件实现（一个译码器或小 ROM）：

```
输入：IR 中的操作码    输出：该指令微程序的入口地址
例：ADD 的操作码 0000 → 入口地址 10 号单元
    SUB 的操作码 0001 → 入口地址 20 号单元
```

红色虚线箭头“依此设计”就是这个意思：**微程序在 CS 里“落位”之后，这张表就定了，硬件按表造出来。** 这一路输出在图里叫“微程序入口（P_0分支地址）”——它是 P_0 位对应的跳转目标

2️⃣ 条件判别测试组合逻辑 —— “MUX 的遥控器”

它决定 MUX 到底放行哪一路。注意看左侧黑框的说明，它的**选择信号由两部分共同生成**：
- **判别测试字段 P_0~P_k**（来自当前从 CS 读出的微指令）；
- **外部状态条件 FB_1~FB_k**（实时反馈的标志位，如零标志 ZF、进位标志 CF）。

关键巧思：**是否跳转 = “测试位置 1” ∧ “条件成立”**，两者同时满足才真正跳

1.3 重点：判别测试字段 P_0~P_k 的三条规则

判别测试字段	下一微地址来自哪里	动作
$P_0 = 1$	微程序入口查找逻辑输出的入口地址	取指完毕，跳到**新指令的微程序开头**
$P_i = 1$（$1 \le i \le k$）且 $FB_i = 1$	预设好的 P_i分支地址	条件成立，微程序**内部跳转**
$P_i = 1$ 但 $FB_i = 0$	下址字段	条件不成立，照常顺序执行
P_0~P_k 全 0	下址字段	纯顺序执行
> **一句话记住：判别测试字段的每一位，对应 MUX 的一路候选地址；全 0 时就选“下址字段”那一路。**		
>		
> （补充：实际设计中通常保证一条微指令至多一个测试位有效；若多位同时为 1 需优先级电路裁决）		

1.4 完整流程 🎬

设定：CS 的 0~4 号单元 = 取指微程序；ADD 微程序入口 = 10 号（占 10~15 号）；SUB 入口 = 20 号

① 上电，μAR = 0 → 执行取指微程序
　0~3号：判别测试全0，每条的下址字段指向下一条 → 顺序走
② 4号（取指最后一条）：P_0 = 1
　此时 IR 里已装好 ADD 的操作码 → 入口查找逻辑输出 10
　MUX 放行"入口地址"这一路 → 时钟脉冲到来，μAR ← 10
③ 10~12号：ADD 微程序顺序执行（P 全 0，下址字段接力）
④ 12号：P_1 = 1，FB_1 接的是进位标志
　CF=1 → 跳到预设的 P_1分支地址 15 号
　CF=0 → 用下址字段，继续走 13 号
⑤ 15号（ADD 微程序最后一条）：P 全 0，下址字段 = 0
　→ μAR ← 0，回到取指微程序，取下一条指令……循环

时钟脉冲的角色：μAR 只在时钟边沿才更新，保证"一拍执行一条微指令"，所有组合逻辑提前算好下一个微地址，边沿一到稳稳打入

1.5 为什么叫"断定法 / 下地址字段法"？

- **下地址字段法**：因为微指令里专门设了一个"下址字段"，直接写明顺序执行的下一地址；
- **断定法**："断定"= 事先确定。微程序一旦设计完，每条微指令"下一步可能去哪"的**全体候选**（入口地址、各分支地址、下址字段）在**设计期就已写死并固化到硬件连线中**，运行时只是"从中挑一个"。这正是它与下节课**计数器法**（μAR 像 PC 那样自动 +1，仅跳转时才改写）的对偶之处

1.6 思考题的答案

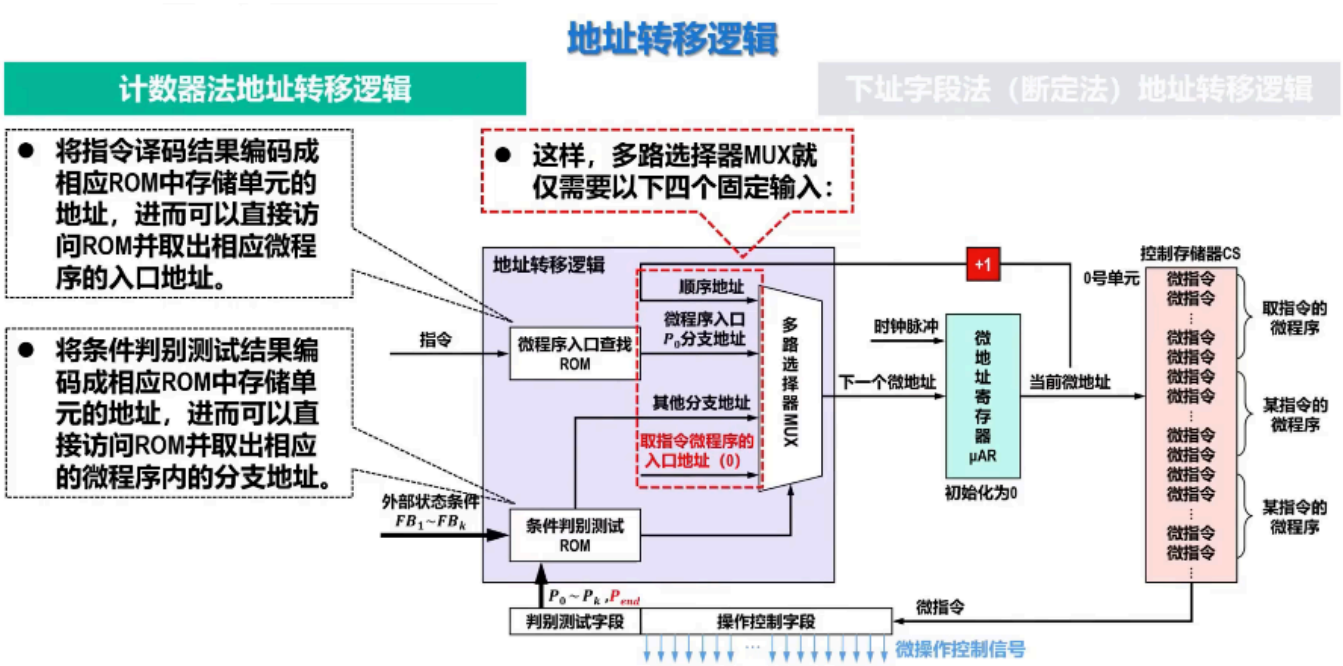
> 若判别测试字段**仅含 P_0 一位**，MUX 简化为几路？选择端来源是什么？
> 用上面的框架推一下：候选地址只剩两路——**微程序入口**（P_0=1 时）和**下址字段**（P_0=0 时）。既然没有 P_1~P_k，条件判别 FB 也用不上了，于是：
> - MUX 退化为 **2 选 1（两路）选择器**；
> - 选择控制端**直接就是 P_0 这一根线**，连"条件判别测试组合逻辑"都可以省掉

1.7 一图流总结



菜谱类比收尾：每做完一步，管家（地址转移逻辑）在三个动作里挑一个——
① 步骤末尾写着"下一步见第 X 步"（下址字段）；
② "如果肉还没烂，回到第 2 步"（P_1~P_k + FB 条件判断）；
③ 客人新点了一道菜，翻目录页找新菜谱（P_0 + 入口查找逻辑）。
MUX 就是管家做出"三选一"决定的那个瞬间——而这个决定的规则，早在写菜谱（微程序设计）时就全部定死了，这就是"断定"二字的含义

二、计数器法



> **断定法把“下一步去哪”写在每条微指令里（下址字段）；计数器法让 μAR 像 PC 一样自动 +1，跳转时才“查表”改道**

2.1 相对下地址字段法：微指令“瘦身”

微指令只剩**判别测试字段 + 操作控制字段**

这是计数器法的标志。因为顺序执行不再需要显式指路（μAR 自动 +1），每条微指令都省掉了一大段地址位。2.4讲精讲该设计

2.2 MUX 的四个固定输入（红色虚线框）

上张图的 MUX 路数 = 下址字段 1 路 + 微程序入口 1 路 + $P_1 \sim P_k$ 各 1 路，**分支点越多路数越多**而计数器法把一切候选地址收拢成**恒定的 4 路**：

档位	来源	何时被选中
① 顺序地址	**+1 增量器**（当前微地址+1）	判别测试全0，默认档
② 微程序入口 P_0分支地址	**微程序入口查找 ROM**	P_0=1（取指完毕，跳新指令）
③ 其他分支地址	**条件判别测试 ROM**	P_1~P_k 有效且条件 FB 成立
④ 取指微程序入口 **(0)**	**固定连线，硬接死的 0**	本指令微程序结束，回取指
注意右上角那个 +1 小方块：它把 μAR 的当前值加一后作为"顺序地址"送回 MUX。μAR 从此兼任**微程序计数器 μPC** 的角色——主存层有 PC+1 取下条指令，控制存层有 μAR+1 取下条微指令，"机器里面还藏着一台小机器"		
### 2.3 两本 ROM：把"查表思想"用到极致		
图中左侧两个黑框注释，说的就是这两本 ROM 的原理，核心都是四个字：**结果当地址**		
#### 1️⃣ 微程序入口查找 ROM——"硬件版跳转表"		
> 原文：将**指令译码结果编码成 ROM 的地址**，直接访问 ROM 取出入口地址		
最简单的实现：**操作码本身就是 ROM 地址**		

```
ADD 的操作码 = 0000  →  查 ROM[0000]  →  里面存的是 10（ADD 微程序入口）
SUB 的操作码 = 0001  →  查 ROM[0001]  →  里面存的是 20
```

写过程序的人都见过这个结构——**`switch-case` 语句编译出来的跳转表（jump table）**，入口 ROM 就是它的硬件版本
对比断定法：断定法里这个功能是"组合逻辑电路"（要手动设计译码逻辑、化简方程）；这里改成 ROM，**只需像填表一样把"操作码→入口地址"写进去。**规整、易设计、用 PROM 还能改写——这是"用存储代替硬连线"的胜利

2️⃣ 条件判别测试 ROM——"岔路口查询手册"

> 原文：将**条件判别测试结果编码成 ROM 的地址**，取出微程序内的分支地址
> 断定法里，各分支地址是"依此设计"**固化在导线**上、直接插进 MUX 的；分支一多，导线就乱成一团。计数器法把它们**全部收进一本 ROM**：

```
ROM 地址 = 哪个测试点（P 字段） + 条件是否成立（FB₁~FBₖ）
例：12 号微指令 P₁=1 且 CF=1 → 编码成某地址 → 查得分支目标 15 号
```

这样无论微程序里有多少岔路口，MUX 上只需要**一根"分支地址"线**——这就是"四个固定输入"得以成立的底气

> 💡 延伸一句：微程序思想的开山鼻祖 Wilkes（1951）的初衷就是"**把控制信息当数据存起来**"
> 这张图里连"分支目标"都搬进了 ROM，正是这一哲学贯彻到底的表现

2.4 为什么必须有“固定输入 0”

图里特意用红色标出**取指令微程序的入口地址（0）**

回忆断定法怎么回取指的：每条微程序**最后一条**的下址字段填 0 就行了——反正有下址字段可写
可现在**下址字段没了**！“回到 0 号单元”这件事没法再写在微指令里，于是只能：
- 把 0 作为**一根固定的硬件连线**常驻 MUX；
- 由判别测试字段给出一个“**本指令微程序到此结束**”的特定指示（如一个专用的结束位/特定编码）来选中它
这就是两种方法的“此消彼长”：字段省下了，就得用固定硬件路径补回来。**天下没有免费的微指令位**

2.5 完整流程

设定：CS 的 0~3 号 = 取指微程序；ADD 入口 = 10 号；12 号处有 P_1 分支（FB_1 接进位标志 CF），查表目标 = 15 号

```
① 上电，μAR = 0 → 执行 0 号取指微指令
② 0~2 号：判别测试全 0 → MUX 选"顺序地址"（当前+1）
   时钟脉冲逐拍打入：μAR ← 1, 2, 3
③ 3 号：P₀ = 1 → MUX 选"入口查找 ROM"
   IR 中是 ADD(0000) → ROM[0000] = 10 → μAR ← 10
④ 10~11 号：全 0，顺序执行（自动 +1）
⑤ 12 号：P₁ = 1
   CF=1 → 条件 ROM 查得 15 → μAR ← 15（跳）
   CF=0 → 不选中该路 → 顺序 μAR ← 13（不跳）
⑥ 15 号（ADD 微程序末条）：出现"结束"指示
   → MUX 选固定输入 0 → μAR ← 0 → 回到取指，取下一条机器指令
⑦ 循环往复……
```

每个时钟边沿，μAR 从 MUX 稳稳载入一拍一个微地址；+1 器和两本 ROM 并行算好所有候选，速度不受影响

三、两法对比 📝

维度	断定法（下址字段）	计数器法
微指令字段	操作控制 + 判别测试 + **下址**	操作控制 + 判别测试（**更短**）
顺序执行	下址字段显式指路	μAR 自动 **+1**
MUX 规模	随分支点增多（k+2 路）	**恒定 4 路**
CS 空间	每条都占地址位 → 浪费	节省
微程序编写	每条都要填下址，繁琐易错	像写顺序程序，只标跳转点，**舒服**
分支灵活性	极高，想跳哪填哪	目标集中存 ROM，改分支要重写 ROM
额外硬件	大 MUX	增量器 + 两本 ROM + 小 MUX
速度	略快	相近（查表与 +1 均并行完成）
选型经验：微程序规整、顺序为主 → 计数器法（写起来像普通编程）；分支密集、需要极致跳转自由 → 断定法。实际处理器的微序列器常把两种思想混合使用		

一句话记忆：计数器法 = μAR 变 μPC 自动 +1，微指令丢掉下址字段换取“瘦身”，代价是把入口地址、分支地址、“回 0”全部外化为 ROM 查表和固定连线——**把复杂性从“指令里”搬到了“硬件表里”**